

Course Outline: Object-Oriented Programming in Python

Title: Object-Oriented Programming in Python **Learnpack:** + Programación Orientada a Objetos **Prerequisite:** Python fundamentals, functions, data structures **Target Audience:** Beginning developers learning to model real-world concepts in code **Total Lessons:** 11 **Format:** Mixed (27% hands-on)

Course Description

This course introduces the fundamental concepts of Object-Oriented Programming (OOP) using Python. Students will learn to think in terms of classes and objects, understanding how to model real-world entities and their behaviors in code. By the end of this course, students will grasp the four core principles of OOP and understand when and why to apply object-oriented design patterns.

Object-oriented programming represents a paradigm shift from procedural thinking to modeling the world through interconnected objects that maintain state and exhibit behaviors. This foundation is essential for understanding modern software architecture, database modeling, and reading existing codebases.

Course Philosophy

This course emphasizes:

- **Estado + Comportamiento Pattern:** Objects should contain both data (attributes) and methods that operate on that data
- **Self Convention:** Understanding that `self` is simply the first parameter, representing "who is acting"
- **Real-world Modeling:** From abstract concepts to concrete implementations
- **Anti-pattern Awareness:** Learning what NOT to do prevents common OOP mistakes

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Classes and Objects Fundamentals	3
02 - Object Behavior and State	2
03 - OOP Core Principles	3
04 - Assessment and Integration	2
05 - Next Steps	1
Total	11

Section 00: Welcome

00.0 Welcome to Object-Oriented Programming 📖

Learning Objectives:

- Understand why OOP matters in modern software development
- Recognize the shift from procedural to object-oriented thinking

- Identify real-world scenarios where OOP provides clear advantages

Content Outline:

- The problem OOP solves: modeling complexity
- From functions to objects: a paradigm shift
- Course journey: theory to practical implementation
- Connection to database modeling and existing codebases

Transitions: This lesson establishes the foundation for understanding classes as blueprints in the next lesson.

Key Principles:

- Objects model real-world entities with state and behavior
- OOP provides structure for managing complex applications
- Understanding OOP is essential for reading modern codebases

Examples:

- A video game with multiple enemies: each enemy needs its own health, position, and actions
 - A user management system: each user has unique data but shares common behaviors
-

Section 01: Classes and Objects Fundamentals

01.0 What is a Class? Understanding the Blueprint 📖

Learning Objectives:

- Define what a class represents in programming terms
- Understand the blueprint metaphor for class design
- Identify the components that make up a class definition

Content Outline:

- Classes as blueprints: the architectural analogy
- Components of a class: attributes and methods
- The constructor method (`__init__`) as the blueprint reader
- Estado + Comportamiento pattern: data and actions together

Transitions: Understanding classes as blueprints leads naturally to creating actual instances (objects) from those blueprints.

Key Principles:

- A class defines the structure and behavior template
- The constructor initializes object state
- Classes combine data (attributes) and actions (methods)
- Self convention: the object needs to know "who is acting"

Examples:

- Class `Enemigo` as blueprint for creating `orco1` , `orco2` , each with unique health and position
- Class `Usuario` defining structure for user accounts with name, email, and login behavior

Anti-patterns Integration:

- **La Clase Estática:** Avoid classes that are just containers for unrelated functions (like a `Matematicas` class with only static methods)
 - **Declarar variables fuera del init:** Always define attributes within the constructor for predictable object state
-

01.1 Classes vs Objects: From Blueprint to Reality 📖

Learning Objectives:

- Distinguish clearly between class definitions and object instances
- Understand how multiple objects can share the same class structure
- Recognize that each object maintains its own independent state

Content Outline:

- The relationship: one class, many objects
- Instance independence: each object has its own memory space
- Object identity: every instance is unique even with identical data
- Lista de Objetos pattern: managing collections of instances

Transitions: With a clear understanding of the class-object relationship, students are ready to write their first class definitions.

Key Principles:

- One class can generate unlimited unique objects
- Each object instance maintains independent state
- Objects of the same class share structure but not data
- Object collections enable powerful data management patterns

Examples:

- One `Enemigo` class creating 100 unique orcs, each with different health values
- A `usuarios = []` list where each element is a `Usuario` instance with unique data

Anti-patterns Integration:

- **Arrays Paralelos:** Avoid separate lists like `nombres = []` and `edades = []` that can become desynchronized
- **Datos Primitivos:** Instead of `coche.rueda1_presion`, `coche.rueda2_presion`, use objects within objects

01.2 Creating Your First Classes and Objects 📖

Challenge Prompt: Create a `Guerrero` class with health and name attributes, then instantiate three different warriors and display their information.

Solution Code:

```
class Guerrero:
    def __init__(self, nombre, vida):
        self.nombre = nombre
        self.vida = vida

    def presentarse(self):
        return f"Soy {self.nombre} y tengo {self.vida} puntos de vida"

# Creating three different warriors
aragorn = Guerrero("Aragorn", 100)
legolas = Guerrero("Legolas", 85)
gimli = Guerrero("Gimli", 120)

print(aragorn.presentarse())
print(legolas.presentarse())
print(gimli.presentarse())
print(f"Total de guerreros creados: 3")
```

Challenge Code:

```
class Guerrero:
    def __init__(self, nombre, vida):
        # Define the warrior's attributes here
        pass

    def presentarse(self):
        # Return a string introducing the warrior
        pass

# Create three different warriors with different names and health
# Print their introductions
# Print the total count of warriors created
```

Section 02: Object Behavior and State

02.0 Methods and Attributes: State + Behavior Pattern

Learning Objectives:

- Understand how methods operate on object state (attributes)
- Implement the Estado + Comportamiento pattern effectively
- Master the `self` parameter and its role in object methods

Content Outline:

- Attributes as object state: what the object "knows"
- Methods as object behavior: what the object "can do"
- The `self` parameter: understanding "who is acting"
- State modification through methods: controlled data changes
- Objetos dentro de Objetos pattern: composition and relationships

Transitions: With a solid understanding of state and behavior, students can now build interactive objects that respond to method calls.

Key Principles:

- Methods should operate on and modify object attributes
- `self` is simply the first parameter representing the acting object
- Objects can contain other objects as attributes
- State changes should happen through methods, not external manipulation

Examples:

- A `Jugador` class with `recibir_daño()` method that modifies `self.vida`
- A `Coche` class with `self.ruedas` attribute containing `Rueda` objects

Anti-patterns Integration:

- **Acceso Total:** Avoid direct attribute modification like `jugador.xp += 100` from outside the class
- **Funciones Externas:** Methods should be inside the class, not external functions that manipulate objects

02.1 Building Interactive Objects

Challenge Prompt: Create a `CuentaBanco` class that can deposit, withdraw, and check balance, ensuring the account cannot go below zero.

Solution Code:

```
class CuentaBanco:
    def __init__(self, titular, saldo_inicial=0):
        self.titular = titular
        self.saldo = saldo_inicial

    def depositar(self, cantidad):
        self.saldo += cantidad
        return f"Depósito exitoso. Nuevo saldo: ${self.saldo}"

    def retirar(self, cantidad):
        if cantidad > self.saldo:
            return "Fondos insuficientes"
        self.saldo -= cantidad
        return f"Retiro exitoso. Nuevo saldo: ${self.saldo}"

    def consultar_saldo(self):
        return f"{self.titular} tiene ${self.saldo}"

# Testing the interactive account
cuenta = CuentaBanco("María", 100)
print(cuenta.consultar_saldo())
print(cuenta.depositar(50))
print(cuenta.retirar(30))
print(cuenta.retirar(200)) # Should show insufficient funds
```

Challenge Code:

```
class CuentaBanco:
    def __init__(self, titular, saldo_inicial=0):
        # Initialize account holder and balance
        pass

    def depositar(self, cantidad):
        # Add money to account and return confirmation message
        pass

    def retirar(self, cantidad):
        # Remove money if sufficient funds, return appropriate message
        pass

    def consultar_saldo(self):
        # Return current balance information
        pass

# Test your account with deposits, withdrawals, and balance checks
```

Section 03: OOP Core Principles

03.0 Abstraction and Encapsulation: Hiding Complexity

Learning Objectives:

- Understand abstraction as hiding implementation details
- Apply encapsulation to control access to object data

- Recognize when and how to use private attributes and methods

Content Outline:

- Abstraction: providing simple interfaces for complex operations
- Encapsulation: controlling access to object internals
- Python's privacy conventions: single and double underscores
- Public vs private: what should be accessible from outside
- Interface design: exposing only what's necessary

Transitions: Understanding abstraction and encapsulation provides the foundation for exploring inheritance relationships.

Key Principles:

- Hide complex implementation behind simple method calls
- Protect object state by controlling access paths
- Design clear, intuitive interfaces for object interaction
- Use privacy conventions to signal internal-only attributes

Examples:

- A `Motor` class where users call `arrancar()` without knowing the internal combustion process
 - A `Calculadora` class where internal calculation steps are hidden from the user
-

03.1 Inheritance: Extending and Reusing Code

Learning Objectives:

- Understand inheritance as an "is-a" relationship
- Implement class hierarchies that promote code reuse
- Override parent methods while maintaining proper inheritance structure

Content Outline:

- Inheritance concept: child classes extending parent capabilities
- The "is-a" relationship test for proper inheritance
- Method overriding: customizing inherited behavior
- `Super()` function: accessing parent class methods
- When to use inheritance vs composition

Transitions: With inheritance established, students can explore how different classes can share common interfaces through polymorphism.

Key Principles:

- Child classes inherit all parent attributes and methods
- Override methods to customize behavior for specific subclasses
- Use inheritance for true "is-a" relationships only
- Avoid inheritance hierarchies deeper than 2-3 levels

Examples:

- `Animal` parent class with `Dog` and `Cat` subclasses
- `Vehiculo` parent with `Coche` and `Motocicleta` children

Anti-patterns Integration:

- **Herencia Profunda:** Avoid deep hierarchies like `Animal -> Mamifero -> Canino -> Perro -> Caniche`
-

03.2 Polymorphism: Many Forms, Same Interface

Learning Objectives:

- Understand polymorphism as "many forms, same interface"
- Implement polymorphic behavior across different class types
- Recognize the power of treating different objects uniformly

Content Outline:

- Polymorphism definition: same method name, different implementations
- Duck typing in Python: "if it walks like a duck..."
- Polymorphic collections: lists of different object types
- Interface consistency: maintaining method contracts across classes

Transitions: With all four OOP principles understood, students are ready to apply them in a comprehensive design challenge.

Key Principles:

- Different classes can implement the same method differently
- Polymorphic code can work with multiple object types
- Consistent interfaces enable flexible, extensible designs
- Focus on what objects can do, not what they are

Examples:

- Different `Instrumento` subclasses all implementing `tocar()` method
 - A drawing application where `Circulo`, `Rectangulo`, and `Triangulo` all have `dibujar()` methods
-

Section 04: Assessment and Integration

04.0 Object-Oriented Design Patterns

Challenge Prompt: Design a complete zoo management system with animals, enclosures, and a zoo class that demonstrates all four OOP principles.

Solution Code:

```
class Animal:
    def __init__(self, nombre, especie):
        self.nombre = nombre
        self.especie = especie
        self._hambre = 0 # Encapsulation: private attribute

    def comer(self):
        self._hambre = 0
        return f"{self.nombre} está comiendo"

    def hacer_sonido(self): # Will be overridden
        return f"{self.nombre} hace un sonido"

class Leon(Animal): # Inheritance
    def hacer_sonido(self): # Polymorphism
        return f"{self.nombre} ruge: ROAAR!"

class Elefante(Animal): # Inheritance
    def hacer_sonido(self): # Polymorphism
        return f"{self.nombre} barrita: PFFRRRR!"
```

```

class Zoo:
    def __init__(self, nombre):
        self.nombre = nombre
        self.animales = [] # Lista de Objetos pattern

    def agregar_animal(self, animal):
        self.animales.append(animal)

    def alimentar_todos(self): # Abstraction
        for animal in self.animales:
            print(animal.comer())

    def concierto_animal(self): # Polymorphism in action
        for animal in self.animales:
            print(animal.hacer_sonido())

# Usage
zoo = Zoo("Safari Park")
simba = Leon("Simba", "León Africano")
dumbo = Elefante("Dumbo", "Elefante Asiático")

zoo.agregar_animal(simba)
zoo.agregar_animal(dumbo)
zoo.concierto_animal()

```

Challenge Code:

```

# Create a zoo management system that demonstrates:
# 1. Inheritance (Animal parent, specific animal children)
# 2. Polymorphism (same method, different implementations)
# 3. Encapsulation (private attributes with public methods)
# 4. Abstraction (complex operations behind simple interfaces)

class Animal:
    def __init__(self, nombre, especie):
        # Your code here
        pass

    def hacer_sonido(self):
        # Base implementation
        pass

class Zoo:
    def __init__(self, nombre):
        # Your code here
        pass

    def agregar_animal(self, animal):
        # Add animal to zoo
        pass

# Create animal subclasses and demonstrate all OOP principles

```

04.1 OOP Mastery Assessment

Learning Objectives:

- Evaluate understanding of OOP core concepts
- Test ability to identify appropriate OOP applications
- Assess comprehension of class vs object relationships

Questions:

- Multiple Choice:** ¿Cuál es la diferencia principal entre una clase y un objeto?
 - a) No hay diferencia, son términos sinónimos
 - b) Una clase es la plantilla, un objeto es una instancia específica de esa plantilla
 - c) Un objeto es más grande que una clase
 - d) Las clases son para datos, los objetos para métodos
- Multiple Choice:** ¿Cuál de estos es un ejemplo del patrón Estado + Comportamiento?
 - a) Una clase Matematicas con métodos sumar() y restar() estáticos
 - b) Una clase Jugador con atributo vida y método recibir_daño() que modifica vida
 - c) Una función que recibe un objeto y modifica sus atributos
 - d) Variables globales que almacenan datos del juego
- Multiple Choice:** ¿Qué representa el parámetro 'self' en un método de Python?
 - a) Una palabra clave mágica que Python requiere
 - b) El primer parámetro que representa el objeto que está actuando
 - c) Una referencia a la clase, no al objeto
 - d) Un parámetro opcional que se puede omitir
- Multiple Choice:** ¿Cuál de estas es una correcta aplicación de herencia?
 - a) Clase Usuario hereda de Clase BaseDatos
 - b) Clase Perro hereda de Clase Animal
 - c) Clase Calculadora hereda de Clase Numero
 - d) Clase Login hereda de Clase Contraseña
- Multiple Choice:** ¿Qué patrón evita el anti-pattern de "Arrays Paralelos"?
 - a) Usar múltiples listas relacionadas por índice
 - b) Lista de Objetos: una sola lista donde cada elemento es un objeto con todos sus datos
 - c) Separar los datos en diferentes archivos
 - d) Usar variables globales para coordinar las listas
- Multiple Choice:** En el contexto de polimorfismo, ¿qué significa "misma interfaz, diferentes implementaciones"?
 - a) Todas las clases deben tener exactamente los mismos métodos
 - b) Diferentes clases pueden tener métodos con el mismo nombre pero comportamiento específico
 - c) Solo se puede usar herencia múltiple
 - d) Los métodos no pueden ser modificados en las clases hijas
- Multiple Choice:** ¿Cuál es el principal beneficio de la encapsulación?
 - a) Hace el código más rápido
 - b) Permite crear más objetos
 - c) Controla el acceso a los datos internos del objeto y protege su integridad
 - d) Elimina la necesidad de usar métodos

Evaluation Criteria:

- **7 correct:** Excelente comprensión de OOP
- **5-6 correct:** Buena comprensión, revisar conceptos específicos
- **3-4 correct:** Comprensión básica, necesita refuerzo
- **0-2 correct:** Requiere revisión completa del material

Section 05: Next Steps

05.0 From Objects to Applications

Content Outline:

- Course recap: From classes to complete OOP systems
- Connection to the bigger picture: OOP in real applications
- Next steps: Advanced OOP concepts and design patterns
- Resources for continued learning: design patterns, SOLID principles
- Encouragement: You now think in objects and classes

Course Recap: You've mastered the fundamental shift from procedural to object-oriented thinking. You understand that classes are blueprints that generate unique object instances, each maintaining its own state while sharing common behaviors. You've implemented the Estado + Comportamiento pattern, learned to control access through encapsulation, extended functionality through inheritance, and created flexible interfaces through polymorphism.

Connection to Bigger Picture: These OOP concepts form the foundation for understanding modern software architecture. The classes you create will become database models, the objects will represent real users and business entities, and the principles you've learned will guide you in building maintainable, scalable applications.

Next Steps:

- Advanced Python OOP: decorators, metaclasses, abstract base classes
- Design patterns: Factory, Observer, Strategy, and more
- SOLID principles for better class design
- OOP in web frameworks: Django models, FastAPI schemas
- Database ORM: translating classes to database tables

Resources:

- Python's official OOP tutorial
- "Design Patterns: Elements of Reusable Object-Oriented Software"
- Practice with real-world modeling exercises
- Open source projects using OOP patterns

Encouragement: You've accomplished a major milestone in your programming journey. Object-oriented thinking is now part of your developer toolkit. Every complex system you encounter can be understood in terms of objects, their states, and their interactions. This foundation will serve you well as you tackle increasingly sophisticated programming challenges.
